

MobFI Operator Handbook

Mobile Filesystem Inspector - complete operator reference

MobFI 1ae717b-dirty

2026-07-31

Contents

Introduction

- Who this is for
- The two frontends
- The trust model
- What MobFI does not do
- How capability depends on device state
- How to read this handbook
- Getting help

Installation

- What you need
- Option 1: prebuilt CLI (fastest)
- Option 2: install script (recommended for a full setup)
- Option 3: build from source manually
- Installing the device tools
- Verify the install
- Windows on ARM64
- Next

First run

- Step 0: confirm the tools
- Step 1: the guided wizard
- Step 2: the same capture with subcommands
- Step 3: read what you pulled
- What “good” looks like
- Environment variables
- Next

Devices

- Detecting
- Columns
- States
- Android connection methods
- iOS connection methods
- No devices detected
- Choosing a device for a workflow
- Next

Enumerating apps

- Listing
- Columns
- Finding the target
- In the GUI

What the app type tells you

Next

Extraction

Android

iOS

Choosing the scope: decision table

Reading the result

Path safety

Practical patterns

In the GUI

Next

Scanning for secrets

Flags

The rule catalog

Known-secret lists

What the scanner skips

Live verification

Interpreting findings

Where the raw values are

In the GUI

Next

Diffing captures

Change kinds

Structural diffing

The canonical workflow

Sequencing captures

Noise

Combining with a scan

In the GUI

Next

Database viewing

Listing and dumping

Finding the databases in a capture

Read-only and evidence integrity

Cell rendering

What to look for

Decoding what you find

In the GUI

Next

Rendering and decoding

Rendering a file

Decoding strings

In the GUI

Next

Keychain and Keystore

The hard limits

iOS: the two routes

Android: rooted device

Reading the output

Revealing values

When nothing is recoverable

Next

Reporting

Output formats

Redaction

Paths in reports

Verification in reports

A complete engagement flow

In the GUI

Next

Console

Android

iOS

Session transcripts

Practical patterns

In the GUI

Next

Updating

Checking

Applying

Automatic notices

Disabling the check

Version pinning for an engagement

Verifying an update

Manual update

Next

Troubleshooting

No devices detected

Device shows `unauthorized` (Android)

Device shows `unpaired` (iOS)

Extraction returns 0 files

Extraction is very slow

`cannot read /data/data/<pkg>`

iOS container extraction denied

- Backup extraction fails or hangs
- Scan finds nothing
- Scan finds far too much
- Verification reports `unknown`
- Keys reports `unavailable`
- Backup keychain decryption fails
- GUI will not start
- GUI shows no app icons
- Console will not connect (iOS)
- adb over TCP will not connect
- Filing a bug

CLI reference

Command summary

- `mfi wizard`
- `mfi detect`
- `mfi apps`
- `mfi extract`
- `mfi scan`
- `mfi diff`
- `mfi report`
- `mfi db`
- `mfi render`
- `mfi decode`
- `mfi keys`
- `mfi doctor`
- `mfi update`
- `mfi version`
- `mfi help`

Environment variables

Exit codes

Appendix

- Repository layout
- On-device paths worth knowing
- Glossary
- Further reading
- Handbook maintenance

Introduction

MobFI (Mobile Filesystem Inspector) inspects the on-device file structures of installed Android and iOS applications. Use it to discover devices, pull an app's private data to your

workstation, hunt for secrets in what you pulled, diff two captures, browse databases, render native file formats, and summarise the result into a report.

This handbook is the complete operator reference. It covers every workflow in both frontends, the constraints that decide which workflow is even possible on a given device, and what to do when a step fails.

Who this is for

You are running MobFI against a device you are authorised to test: your own handset, a lab device, a client device inside an agreed scope. MobFI does not bypass platform security. Everything it reaches, it reaches through documented mechanisms (`adb` , `libimobiledevice` , backups) that the device owner has enabled by unlocking the device, authorising the host, and (where required) rooting or jailbreaking it.

Prior knowledge assumed: a terminal, basic Android/iOS concepts (package name vs bundle id, USB debugging, device pairing). No Go knowledge required unless you plan to build from source.

The two frontends

One shared core library in Go (`internal/`) holds every piece of logic: device I/O, extraction, scanning, diffing, database access, rendering, reporting. Two thin frontends drive it.

Frontend	Binary	Best for
CLI	<code>mfi</code>	Scripting, remote/headless hosts, reproducible captures, piping into other tools
Desktop GUI	<code>mfi-gui</code>	Exploration, browsing extracted trees, reading rendered files, interactive triage

Every capability is available from both, because both call the same core methods. The CLI is cgo-free and ships as a prebuilt binary; the GUI is a [Wails](#) app (Go backend, dependency-free HTML/JS/CSS UI) that you build locally because it needs per-OS WebKit/WebView toolchains.

When this handbook shows a workflow, it gives the CLI invocation first (precise, copy-pasteable) and the GUI equivalent second.

The trust model

Read this section before pointing MobFI at a device you did not personally provision.

The device is untrusted input. Everything that crosses from the device into MobFI is attacker-controlled if the device is compromised or if you are examining a malicious app:

- File and directory names returned by on-device enumeration.
- File contents: SQLite databases, property lists, JSON, images, PDFs, source-looking text.
- App metadata: bundle ids, icon resource names, `dumpsys` output.
- Backup manifests, when you use the iOS backup extraction scope.

MobFI treats that data as hostile, but the guarantee is not absolute: parsing arbitrary attacker-controlled formats is inherently risky. Practical guidance:

- Extract into a **fresh, dedicated directory** per capture. Do not reuse a destination across engagements or across devices.
- Prefer a **disposable analysis VM** when examining known-malicious apps.
- Treat the extracted tree as evidence: do not execute anything out of it. The GUI's "Open externally" hands a file to your operating system's default handler, which will happily run an executable.

Your workstation is the thing being defended. MobFI's job is to copy data off a device without letting the device reach back into your host.

Extracted data is client data. A capture from a real engagement contains credentials, tokens, personal information, and session material belonging to whoever owns the device. Handle the destination directory with the same care as any other evidence artifact: encrypted volume, retention policy, secure destruction.

What MobFI does not do

- **It does not exploit.** No jailbreak, no root exploit, no sandbox escape. If a device is stock and locked, MobFI's reach is limited to what backups and debuggable apps expose.
- **It does not modify the device.** Extraction is read-only from the device's perspective. The one exception is the iOS backup scope, which asks the device to produce a backup (a normal, user-initiated operation).
- **It does not decrypt what the platform will not release.** Hardware-backed Android Keystore private keys and iOS Secure Enclave keys are non-exportable by design. MobFI inventories them; it cannot dump them.
- **It does not phone home.** The only outbound network traffic is the update check (which you can disable with `MFI_NO_UPDATE_CHECK`) and the opt-in secret verification described in the scanning chapter.

How capability depends on device state

More than anything else, what you can extract is decided by the device's state. This table is the single most useful thing in the handbook; the extraction chapter expands each row.

Platform	Device state	App type	What you can reach
Android	Any (USB debugging on)	Debuggable (<code>android:debuggable="true"</code>)	Full private data dir via <code>run-as</code>
Android	Rooted	Any	Full private data dir via <code>su</code>
Android	userdebug/eng build	Any	Full private data dir via the shell user
Android	Stock, locked	Non-debuggable	Nothing private. Public storage only
iOS	Any (paired + trusted)	File-sharing enabled	The app's <code>Documents</code> directory
iOS	Any (paired + trusted)	Dev-signed / debug build	The full app container
iOS	Any (paired + trusted)	Any, incl. App Store	App data via an encrypted device backup
iOS	Jailbroken	Any	Full container, plus keychain via SSH
iOS	Simulator (macOS)	Any installed	Container directly off the host filesystem

If a workflow in this handbook fails, check this table first. Most “MobFI cannot see my app's data” reports are a device-state mismatch, not a bug.

How to read this handbook

- **New to MobFI:** read Installation, then First Run, then stop. That is enough to complete a capture end to end.
- **Doing a specific job:** jump to the chapter for that job. Each is self-contained and states its own prerequisites.
- **Something broke:** Troubleshooting, then the chapter for the workflow that broke.
- **Looking up a flag:** the CLI Reference chapter lists every command and every flag.

Conventions used throughout:

- `$ command` denotes a shell prompt on your workstation. The `$` is not part of the command.
- `<angle-brackets>` mark a value you substitute.
- Blocks labelled **OPSEC** flag behaviour that is observable on the device, on the network, or by a third party.
- Blocks labelled **Evidence** flag behaviour that matters for chain of custody.

Getting help

- `mfi help` lists every command.
- `mfi <command> -h` lists the flags for one command.
- `mfi doctor` reports which external tools are installed and how to install the missing ones.
- The repository `README.md` covers build and install specifics that change more often than this handbook.

Installation

MobFI itself is a single static binary. What takes setup is the external tooling it drives: `adb` for Android, the `libimobiledevice` suite for iOS. MobFI runs without them, but a platform whose tools are missing simply reports no devices.

What you need

Component	Needed for	Notes
<code>mfi</code> binary	Everything	Prebuilt download, or <code>make build</code>
<code>adb</code>	All Android work	Android platform-tools
<code>idevice_id</code> , <code>ideviceinfo</code>	iOS device discovery	<code>libimobiledevice</code>
<code>afcclient</code>	iOS container/documents extraction	<code>libimobiledevice</code>
<code>ideviceinstaller</code>	iOS app listing	Separate package on some platforms
<code>idevicebackup2</code>	iOS backup extraction (optional)	Needed only for the <code>backup</code> scope
<code>iproxy</code> , <code>ssh</code>	iOS Console over USB (optional)	Jailbroken devices
<code>aapt</code>	Real Android icons/names in the GUI (optional)	Android SDK build-tools
<code>xcrun</code> , <code>plutil</code>	iOS Simulator support (macOS only)	Ships with Xcode / macOS
Go 1.23+	Building from source	Not needed for prebuilt CLI
Wails CLI + Node + C toolchain	Building the GUI	See the GUI section

Nothing here is mandatory up front. Install the tools for the platform you are actually testing, then run `mfi doctor` to confirm.

Option 1: prebuilt CLI (fastest)

Download the binary for your platform from the [latest release](#). Binaries are cgo-free and version-stamped.

```
# macOS / Linux
$ chmod +x mfi_v1.0.0_darwin_arm64
$ shasum -a 256 -c SHA256SUMS.txt --ignore-missing
$ ./mfi_v1.0.0_darwin_arm64 version
```

Verify the checksum before running it. `SHA256SUMS.txt` is published on the same release page.

On macOS, Gatekeeper quarantines downloaded binaries. Clear it:

```
$ xattr -d com.apple.quarantine mfi_v1.0.0_darwin_arm64
```

Rename it to `mfi` and put it somewhere on your `PATH`:

```
$ mkdir -p ~/.local/bin
$ mv mfi_v1.0.0_darwin_arm64 ~/.local/bin/mfi
```

The desktop GUI is not shipped prebuilt: it needs per-OS cgo and WebKit toolchains. Build it with the install script below.

Option 2: install script (recommended for a full setup)

One command resolves everything (Go toolchain, Wails CLI and its native GUI toolchain, `adb`, `libimobiledevice`) and builds both the CLI and the GUI. Re-running it is safe; anything already present is skipped. It puts `mfi` on your `PATH`.

```
$ git clone https://github.com/integrise/MobFI.git
$ cd MobFI

# macOS / Linux
$ ./scripts/install.sh           # or: make setup

# Windows (PowerShell)
$ powershell -ExecutionPolicy Bypass -File scripts\install.ps1
```

Useful flags on both scripts:

Flag (sh)	Flag (ps1)	Effect
<code>--cli-only</code>	<code>-CliOnly</code>	Skip the GUI build
<code>--gui-only</code>	<code>-GuiOnly</code>	Skip the CLI build
<code>--no-runtime-tools</code>	<code>-NoRuntimeTools</code>	Do not install <code>adb</code> / <code>libimobiledevice</code>
<code>--launch cli\ gui</code>	<code>-Launch cli\ gui</code>	Run the app once built
(n/a)	<code>-NoShortcuts</code>	Skip Start Menu / Desktop shortcuts

Per-OS behaviour of the installer

macOS uses Homebrew (installing it if missing) and copies the built `MobFI.app` into `/Applications`.

Linux auto-detects `apt` / `dnf` / `pacman` / `zypper` and installs the GTK3 + WebKit2GTK build packages. If the system Go is older than 1.23 it fetches a current one from go.dev.

Windows uses `winget` for Go, platform-tools (`adb`), and WebView2, and bootstraps `scoop` to get `gcc` (Wails links WebView2 through `cgo`, so the GUI build needs a C compiler; the CLI is `cgo-free` and does not). For iOS it downloads prebuilt `libimobiledevice` binaries and installs Apple Mobile Device Support via `winget`, which provides the `usbmuxd` service `libimobiledevice` needs. `scoop` bootstrapping requires a **non-elevated** PowerShell.

Option 3: build from source manually

```
$ make build      # compiles the CLI to ./bin/mfi
$ make test       # run the test suite
$ make vet        # go vet
```

Put it on your `PATH` by symlink, so a later `git pull` + rebuild is picked up automatically:

```
$ mkdir -p ~/.local/bin && ln -sf "$PWD/bin/mfi" ~/.local/bin/mfi
```

Building the GUI

In addition to Go, you need:

- The Wails CLI: `go install github.com/wailsapp/wails/v2/cmd/wails@latest`
- Node.js and a C toolchain / platform webview. On macOS that is the Xcode Command Line Tools.

Run `wails doctor` to check your toolchain, then build from `cmd/mfi-gui`. The frontend under `frontend/dist` is vanilla HTML/JS/CSS and needs no npm build step.

Installing the device tools

If you skipped the installer or want to add a platform later:

Android (`adb`)

```
# macOS
$ brew install --cask android-platform-tools

# Debian / Ubuntu
$ sudo apt install android-tools-adb

# Windows
$ winget install Google.PlatformTools
```

iOS (libimobiledevice)

```
# macOS
$ brew install libimobiledevice
$ brew install ideviceinstaller

# Debian / Ubuntu
$ sudo apt install libimobiledevice-utils
$ sudo apt install ideviceinstaller
```

On Windows there is no winget or scoop package for the suite. Run `scripts\install.ps1`, which pulls the core tools (`idevice_id`, `ideviceinfo`, `afcclient`) from the `jrjr/libimobiledevice-windows` release bundle and `ideviceinstaller` from the `imobiledevice-net/win-x64` bundle. If a download fails, fetch the zip manually, extract it, and add the folder to your `PATH`.

Optional extras

```
# iOS Console over USB (jailbroken devices)
$ brew install libusbmuxd          # macOS: provides iproxy
$ sudo apt install libusbmuxd-tools # Debian/Ubuntu

# Real Android app icons and names in the GUI
$ brew install --cask android-commandlinetools
$ sdkmanager "build-tools;34.0.0"
```

Verify the install

```
$ mfi version
$ mfi doctor
```

`mfi doctor` prints a table of every tool MobFI knows how to use, whether it was found, where, and how to install it if not:

```
MobFI dependency check (darwin/arm64)
```

STATUS	TOOL	PURPOSE	LOCATION / INSTALL
ok	adb	Android device access (detect, extract, ...)	/opt/homebrew/bin/adb
ok	idevice_id	iOS device discovery (libimobiledevice)	/opt/homebrew/bin/idevice_id
MISSING	ideviceinstaller	iOS installed-app listing	-> brew install ideviceinstaller
optional	idevicebackup2	iOS backup-based extraction	-> brew install libimobiledevice

`MISSING` marks a core tool: some platform workflow is unavailable without it. `optional` marks a nice-to-have whose absence disables one feature, not a platform.

Machine-readable form for scripts:

```
$ mfi doctor -json
```

Windows on ARM64

Two constraints worth knowing before you plan an engagement around an ARM Windows host:

- scoop's `gcc` is x86-64, so the installer builds the GUI as `windows/amd64` and it runs under the OS's x64 emulation. The cgo-free CLI builds natively as `arm64`.
- **iOS over USB does not work.** Apple's Mobile Device USB driver is x64 kernel-mode, and Windows on ARM cannot load x64 kernel drivers, so iPhones may not enumerate even with the service installed. Use a Mac or an x64 host for iOS. Android is fine.

Next

Continue to First Run for an end-to-end capture.

First run

This chapter takes you from a freshly installed MobFI to a completed capture: device detected, app extracted, tree scanned, report written. Budget fifteen minutes.

Use an Android device with USB debugging enabled and a debuggable test app installed, or an emulator. That is the shortest path to a successful first capture. iOS needs more setup, covered in the Devices and Extraction chapters.

Step 0: confirm the tools

```
$ mfi doctor
```

For an Android-only first run, you need `adb` to show `ok`. Ignore iOS rows for now.

Step 1: the guided wizard

Running `mfi` with no arguments starts the wizard. It walks the same core code the subcommands use, so nothing you learn here is throwaway.

```
$ mfi
```

The wizard runs five steps:

1. **Detect devices.** Lists everything reachable and asks you to pick one. `r` re-detects (useful after plugging in a cable or accepting the USB debugging prompt).
2. **List apps.** Shows user-installed apps by default. `a` widens the list to include system apps.
3. **Extract.** Asks for a destination directory. On iOS it also asks which AFC scope to use.
4. **Scan and/or diff.** Offers a secrets scan of what you just pulled, and optionally a diff against a second capture.
5. **Report.** Prints a summary and offers to write it to a file.

Type `q` (or press Ctrl-D) at any prompt to quit cleanly.

A leading `~` in any path prompt expands to your home directory.

What the wizard looks like

```
MobFI guided wizard - type `q` (or Ctrl-D) at any prompt to quit.
Advanced users can run subcommands directly; see `mfi help`.

Step 1 - Detecting devices...

  1. emulator-5554          Pixel_7_API_34 (android/emulator, ready)
Select a device [1-1], [r]e-detect, or [q]uit: 1

Step 2 - Listing apps on Pixel_7_API_34...

  1. Example Target          com.example.target
Select an app [1-1], [a]ll (incl. system), or [q]uit: 1

Step 3 - Extract com.example.target
Destination directory: ~/captures/target-baseline
extracted 148 file(s), 2317884 byte(s) to /home/op/captures/target-baseline

Step 4 - Scan and/or diff
Scan the extracted tree for secrets? [Y/n]: y
Known-secrets file to also search (optional, blank to skip):
scanning...
3 finding(s).
Diff this capture against another extracted root? [y/N]: n

Step 5 - Report
...
```

The wizard's report is always redacted. To include raw secret values, use `mfi report -show-secrets` (see the Reporting chapter).

Step 2: the same capture with subcommands

Everything the wizard did maps to a subcommand. Once you know the device serial and bundle id, this is faster and scriptable.


```
# 1. What is connected?
$ mfi detect
ID            NAME            PLATFORM  TRANSPORT  STATE
emulator-5554  Pixel_7_API_34  android  emulator   ready

# 2. What is installed?
$ mfi apps -device emulator-5554
BUNDLE ID      NAME            VERSION  DATA PATH            INSTALL PATH
com.example.target  Example Target  1.4.2    /data/data/com.example.target  /data/app/...

# 3. Pull it
$ mfi extract -device emulator-5554 \
  -app com.example.target \
  -out ~/captures/target-baseline

# 4. Look for secrets
$ mfi scan -root ~/captures/target-baseline

# 5. Summarise
$ mfi report -root ~/captures/target-baseline -out ~/captures/target-report.json
```

Step 3: read what you pulled

Two ways to explore the capture.

Render a single file from the CLI:

```
$ mfi render -file ~/captures/target-baseline/shared_prefs/auth.xml
```

MobFI picks a renderer by content, not by extension: SQLite, JSON, property list (binary and XML), generic XML, plain text, and a hex dump as the catch-all.

Browse the whole tree in the GUI:

```
$ mfi-gui
```

Open the Render tab, point it at the capture directory, and click through the file browser. Databases open in the Database tab.

What “good” looks like

After a successful first run you should have:

- A destination directory that mirrors the app’s on-device tree.
- A non-zero file count reported by the extract step.
- A scan that completed (zero findings is a perfectly good result).
- A report file you can open.

If the extract reported **0 files**, the device state does not allow reading that app's private data. That is the single most common first-run outcome. Go to the Extraction chapter and check your app and device against the capability table.

Environment variables

Variable	Effect
<code>MFI_NO_UPDATE_CHECK</code>	Set to any value to disable the update check at startup
<code>MFI_BACKUP_PASSWORD</code>	iOS backup password, used by <code>mfi keys -backup</code> instead of <code>-password</code>
<code>MFI_UPDATED</code>	Set internally after a self-update re-exec; do not set manually

Prefer `MFI_BACKUP_PASSWORD` over `-password` on shared hosts: command-line arguments are visible to other users through the process table.

Next

- Devices: connection methods, states, and what to do when a device does not appear.
- Extraction: which scope or mechanism to use for your target.

Devices

Everything starts with a device MobFI can see. This chapter covers detection, the connection methods for each platform, what each reported state means, and how to fix a device that does not appear.

Detecting

```
$ mfi detect
```

ID	NAME	PLATFORM	TRANSPORT	STATE
emulator-5554	Pixel_7_API_34	android	emulator	ready
39GH2A0MC1	SM-G991B	android	usb	ready
00008030-0011	Test iPhone	ios	usb	ready

MobFI runs three detectors and merges the results:

Detector	Finds	Requires
adb	Android over USB, emulator, adb-over-TCP	adb
libimobiledevice	iOS over USB and network	idevice_id , ideviceinfo
simctl	iOS Simulators (macOS)	xcrun

A detector that fails does not suppress the others. If `adb` is missing, iOS devices still list, and the error naming the failed detector is reported alongside the devices that were found. This is deliberate: a partial answer beats no answer.

In the GUI, the Devices tab polls on a timer, so plugging a cable updates the list without a manual refresh.

Columns

Column	Meaning
ID	adb serial (Android) or UDID (iOS). This is what you pass to <code>-device</code>
NAME	Human-friendly label reported by the device
PLATFORM	<code>android</code> or <code>ios</code>
TRANSPORT	<code>usb</code> , <code>tcp</code> , <code>emulator</code> , or <code>simulator</code>
STATE	Readiness, see below

States

State	Meaning	What to do
<code>ready</code>	Connected and authorised. Extraction can proceed	Nothing
<code>unauthorized</code>	Android: the USB debugging prompt has not been accepted	Unlock the device, accept the prompt, tick “always allow”
<code>offline</code>	adb sees the device but cannot talk to it	Replug, or <code>adb kill-server && adb start-server</code>
<code>unpaired</code>	iOS: the host is not trusted by the device	Unlock the device, replug, tap “Trust”, enter the passcode

MobFI reports adb’s raw `device` state as `ready`, because “device” as a state next to a column called PLATFORM reads as noise.

Android connection methods

USB

The default. Enable Developer Options and USB debugging on the handset, plug it in, and accept the prompt.

```
$ mfi detect
```

If the state is `unauthorized`, the prompt was dismissed or never appeared. Unlock the screen and replug. If the prompt still does not appear, revoke existing authorisations on the device (Developer Options, “Revoke USB debugging authorisations”) and replug.

Emulator

An Android emulator registers with `adb` automatically. It shows up with transport `emulator` and an `emulator-NNNN` serial. Emulators are the easiest way to practise: everything is debuggable and rooting is a non-issue.

adb over TCP (wireless debugging)

Two distinct steps on Android 11 and later, and people routinely conflate them.

Pairing happens once, using the pairing dialog's host:port and its six-digit code:

```
$ adb pair 192.168.1.42:37129 123456
```

In the GUI, use the Devices tab's Pair control.

Connecting uses a *different* port, shown on the Wireless debugging screen itself:

```
$ adb connect 192.168.1.42:5555
$ mfi detect
```

In the GUI, use the Connect control.

Gotchas that account for most wireless-debugging failures:

- **The pairing values expire fast** and change every time the dialog is reopened. Read them and type them promptly.
- **The pairing port is not the connect port.** Pairing succeeds, then `adb connect` to the same port fails. Use the port from the main Wireless debugging screen.
- **Same network, no client isolation.** Guest Wi-Fi and many corporate SSIDs block client-to-client traffic.
- **VPNs hijack RFC1918 routes.** If a VPN captures the route to the phone's address, the connection silently fails. Check with `route -n get <ip>` on macOS or `ip route get <ip>` on Linux.

OPSEC: adb over TCP exposes an authenticated debugging channel on the local network for as long as it is enabled. Turn wireless debugging off on the device when you are done.

iOS connection methods

USB

Pair and trust first:

1. Unlock the device.

2. Plug it in.
3. Tap **Trust** on the “Trust This Computer?” prompt.
4. Enter the device passcode.

Then:

```
$ mfi detect
$ ideviceinfo -u <udid> | head      # sanity check libimobiledevice itself
```

A device in state `unpaired` has not completed that flow. Replug with the screen unlocked. If the prompt never appears, reset the trust relationship on the device (Settings, General, Transfer or Reset, Reset Location & Privacy) and replug.

On Linux, `usbmuxd` must be running. On Windows, the equivalent comes from Apple Mobile Device Support, which `scripts\install.ps1` installs via winget.

Network

`libimobiledevice` can reach a paired device over the network when the device has network pairing enabled. It appears with transport `tcp`. Pair over USB first; network detection reuses the existing pairing record.

Simulator (macOS only)

Booted iOS Simulators are detected through `xcrun simctl` and appear with transport `simulator`. Simulators are the friendliest iOS target: the container lives on your host filesystem, so MobFI copies it directly instead of going through AFC, and every app is reachable.

```
$ xcrun simctl list devices booted  # confirm a simulator is running
$ mfi detect
```

If simulator detection fails with an `xcrun` error, `xcode-select` is probably pointing at the Command Line Tools rather than a full Xcode. MobFI works around this by looking for `simctl` inside `/Applications/Xcode.app` (and `Xcode-beta.app`) directly, but a correct `xcode-select -s` is the cleaner fix.

No devices detected

Work through this in order.

1. **Is the tool installed?** `mfi doctor`. No `adb` means no Android devices, ever.
2. **Does the underlying tool see it?** This separates a MobFI problem from an environment problem:

```
$ adb devices -l          # Android
$ idevice_id -l           # iOS
$ xcrun simctl list devices booted  # Simulator
```

If the native tool does not see the device, MobFI cannot either. Fix the environment first.

3. **Is the screen unlocked?** Both platforms gate authorisation prompts behind an unlocked screen.
4. **Is it a charge-only cable?** A surprising number of cables carry power but not data. Swap it.
5. **Restart the daemon.** `adb kill-server && adb start-server`. On Linux, `sudo systemctl restart usbmuxd` for iOS.
6. **Check udev rules** on Linux for Android: without them, `adb` sees the device as `no permissions`.

Choosing a device for a workflow

Once detected, pass the `ID` column to any command that needs a device:

```
$ mfi apps    -device <id>
$ mfi extract -device <id> -app <bundle-id> -out <dir>
$ mfi keys    -device <id> -platform android -state rooted
```

The ID must match exactly. `mfi` resolves it against a fresh detection pass and errors with “device not found; run `mfi detect` ” if it cannot.

Next

- Apps: enumerate what is installed and pick a target.
- Extraction: pull the target’s data.

Enumerating apps

Before extracting, you need the target's exact bundle id. A wrong or approximate id is the second most common cause of an empty extraction (after device state).

Listing

```
$ mfi apps -device <device-id>
```

BUNDLE ID	NAME	VERSION	DATA PATH	INSTALL PATH
com.example.target	Example Target	1.4.2	/data/data/com.example.target	/data/app/
~~ab12../base.apk				
com.example.other	Other App	0.9.1	/data/data/com.example.other	/data/app/
~~cd34../base.apk				

By default only user-installed apps are listed. Include the system apps too:

```
$ mfi apps -device <device-id> -all
```

System apps are excluded by default because a stock handset lists several hundred of them and the target is almost never among them. Include them when you are auditing preinstalled software, chasing an OEM component, or the app you want genuinely ships in the system image.

Columns

Column	Android source	iOS source
BUNDLE ID	Package name from the package manager	CFBundleIdentifier
NAME	Application label	CFBundleDisplayName
VERSION	versionName	CFBundleShortVersionString
DATA PATH	/data/data/<pkg>	App container path
INSTALL PATH	APK path under /data/app	Bundle path

DATA PATH is what extraction targets. INSTALL PATH points at the installed binary/APK, which is useful when you want to pull the app package itself rather than its data.

Finding the target

The bundle id is rarely the app's display name. Filter the list:

```
# Android: everything from one vendor
$ mfi apps -device <id> | grep -i acme

# By display name when you only know the marketing name
$ mfi apps -device <id> | grep -i "mobile banking"
```

If the app is not in the default list, try `-all`. If it is still absent, it is not installed on that device, or the platform tool cannot enumerate it (see Troubleshooting).

In the GUI

The Apps tab adds interactive affordances the CLI cannot:

- A **search box** filtering by bundle id or name as you type.
- An **Include system apps** checkbox, matching `-all`.
- **Sortable and resizable columns**.
- **Per-row Copy** for the bundle id.
- **Real app icons, names, and versions**, resolved lazily from each APK with `aapt` (from the Android SDK build-tools). Without `aapt`, the GUI falls back to a monogram avatar and a name derived from the bundle id. This is cosmetic: extraction does not need `aapt`.

Clicking a row opens a details panel.

Android (from `dumpsys` package): version, SDK levels, ABI, first-install and last-update timestamps, on-disk sizes, package flags, APK signing version, data and code paths, and the full permission list.

iOS (from `ideviceinstaller`): version, application type, minimum iOS version, signer identity, paths, and entitlements.

The permission and entitlement lists are the fastest way to judge whether an app is worth extracting: an app holding `READ_EXTERNAL_STORAGE`, `ACCESS_FINE_LOCATION`, and a keychain sharing entitlement is a richer target than a static utility.

What the app type tells you

On iOS the details panel's application type decides whether AFC extraction will work at all:

Type	Meaning	Container reachable over AFC?
User	App Store or enterprise-signed	No, unless file sharing is enabled
Developer	Dev-signed / debug build	Yes
System	Apple system app	No

On Android the equivalent signal is the `pkgFlags` list in the details panel: a package carrying `DEBUGGABLE` can be read with `run-as` on any device, no root required.

Both are covered in detail in the Extraction chapter.

Next

Extraction: pull the target's private data.

Extraction

Extraction mirrors an app's on-device file tree into a local directory. This is the core operation: everything downstream (scanning, diffing, database browsing, rendering) works on what you pull here.

```
$ mfi extract -device <device-id> \
              -app <bundle-id> \
              -out <destination-dir> \
              [-scope container|documents|backup]
```

Flag	Required	Meaning
-device	yes	Device ID from mfi detect
-app	yes	Package name (Android) or bundle id (iOS)
-out	yes	Local destination directory, created if absent
-scope	iOS only	container (default), documents , or backup

Progress streams to stderr, so stdout stays a clean summary you can redirect:

```
$ mfi extract -device <id> -app com.example.target -out ./cap > summary.txt
```

Evidence: extract into a fresh directory per capture. Reusing a destination mixes two devices' data into one tree and destroys the provenance of everything in it.

Android

Android extraction targets `/data/data/<package>`, the app's private data directory. MobFI probes three access mechanisms in order and uses the first that can actually read the directory:

1. **run-as <package>** runs commands as the app's own uid. Works on any device, no root, but only for a **debuggable** app (one built with `android:debuggable="true"`). This is the normal path for testing your own builds.
2. **su -c as root** on a rooted device. Reaches any app's data regardless of debuggability. A Magisk or superuser prompt may appear on the device; approve it.
3. **A plain shell**, for devices where the shell user already has access (`adb root` on userdebug/eng builds).

The probe checks command *output*, not just exit status: `adb` does not reliably propagate a remote command's exit code, so a directory that does not exist can otherwise look like success and produce a silent empty extract.

If none of the three can read the directory, extraction fails with:

```
cannot read /data/data/com.example.target: check the package name is
exactly right and the app is installed; a non-debuggable app needs
root (approve the su/superuser prompt on the device)
```

That message is almost always literally correct. Check the package name first (copy it from `mfi apps`), then device state.

Transfer mechanism

Where the device has `tar` (toybox, Android 6 and later), MobFI streams the whole tree in a single `adb exec-out tar` process rather than one `cat` per file. That is dramatically faster on trees with many small files, which describes almost every app.

If the device has no `tar`, or the stream produces nothing, MobFI falls back automatically to per-file copying. You do not choose; there is no flag. A capture that took the fallback path simply takes longer.

Symlinks, sockets, and FIFOs are skipped rather than followed. That avoids `cat` blocking forever on a special file and avoids following a symlink out of the app's tree.

ios

iOS offers three scopes, and picking the right one is the whole game.

`container` (default)

The full app container over AFC house arrest.

```
$ mfi extract -device <udid> -app com.example.target -out ./cap
```

Works for **dev-signed and debug builds**. App Store apps deny it. When the container scope fails, MobFI appends guidance to the error rather than leaving you guessing:

```
(full container access needs a dev-signed/debug app or a jailbreak;
try the 'documents' scope, or 'backup' scope to pull a production
app's data)
```

documents

Just the app's `Documents` directory, over the same AFC mechanism.

```
$ mfi extract -device <udid> -app com.example.target -out ./cap -scope documents
```

Works more broadly than `container`, because any app that enables file sharing (`UIFileSharingEnabled`) vends its `Documents` directory to the host. You get less data, but you get it from apps that would otherwise refuse entirely. Try this before reaching for a backup.

backup

Reconstructs the app's data from a **full device backup** made with `idevicebackup2`.

```
$ mfi extract -device <udid> -app com.example.target -out ./cap -scope backup
```

This is the only way to reach a production App Store app's private data on a stock, non-jailbroken device. MobFI drives `idevicebackup2` to produce a backup, then reads the backup's `Manifest.db` and reconstructs just the target app's domains into your destination: the app's own `AppDomain`, plus any related domain referencing the bundle id (app groups, extensions and plugins, keychain-shared containers).

What to expect:

- **It is slow.** A full backup of a device with photos and media can run for tens of minutes and consume tens of gigabytes of temporary space before reconstruction begins. MobFI estimates the size up front from `ideviceinfo`'s disk-usage domain.
- **It needs free disk.** Budget for the whole device backup, not just the app's share.
- **Symlinks in the backup are recorded, not recreated**, since a backup symlink points at an on-device path that does not exist locally.
- **Encrypted backups are required for keychain data.** The extraction itself works either way, but see the Keys chapter: the keychain is only present in encrypted backups.

OPSEC: a backup is a user-visible operation. It appears in the device's backup history and can trigger "iPhone backup" indications on the device. Do not use this scope when a covert capture matters.

Simulator

For a booted iOS Simulator (`transport simulator`), MobFI bypasses AFC entirely and copies the container straight off the host filesystem. No scope applies, every installed app is reachable, and it is fast. This is the best target for building familiarity.

Choosing the scope: decision table

Your situation	Scope
iOS Simulator	(automatic)
Your own dev/debug build on a device	container
Jailbroken device	container
App Store app, file sharing enabled	documents
App Store app, need everything	backup
Documents scope returned too little	backup

Reading the result

```
extracted 148 file(s), 2317884 byte(s) to /home/op/captures/target
skipped 2 path(s):
  /data/data/com.example.target/cache/locked.db: Permission denied
  /data/data/com.example.target/files/sock: unsupported entry type
```

Three things to check every time:

- **File count is non-zero.** Zero files means the mechanism could not read the tree. Re-check bundle id and device state.
- **Skipped paths.** MobFI records what it could not copy instead of silently dropping it. Permission denials on individual files are normal (some app caches are locked while the app is running); wholesale denials mean the wrong access mechanism was used.
- **Byte count is plausible.** A 4 KB “capture” of an app you know holds a large database means you got a skeleton, not the data.

Skipped entries are recorded in the JSON report too, so a report consumer can see the gap rather than assume a clean tree.

Path safety

Device-supplied names are not trusted. MobFI applies two guards:

- **Destination containment.** A device path that would resolve outside the destination directory is refused and recorded as skipped, rather than written.
- **Host-legal filenames.** Names that are legal on the device but illegal on your host (a Firebase config file containing `%` on Windows, for example) are percent-encoded per component so the tree still writes, with directory structure preserved.

Practical patterns

Baseline and post-action captures, the setup for diffing:

```
$ mfi extract -device <id> -app <bundle> -out ./cap-loggedout
# ... log into the app on the device ...
$ mfi extract -device <id> -app <bundle> -out ./cap-loggedin
$ mfi diff -a ./cap-loggedout -b ./cap-loggedin
```

Capture then immediately scan:

```
$ mfi extract -device <id> -app <bundle> -out ./cap && mfi scan -root ./cap
```

Several apps in one pass:

```
$ for pkg in com.example.a com.example.b com.example.c; do
  mfi extract -device <id> -app "$pkg" -out "./caps/$pkg"
done
```

In the GUI

The Extract tab exposes the same options: device picker, app picker, destination browser, and (on iOS) a scope selector. Progress shows as a live file and byte count with a cancel button. Cancelling stops the transfer; files already written stay on disk.

Next

- Scanning: find secrets in what you pulled.
- Diffing: compare two captures.

Scanning for secrets

The scanner walks an extracted tree and reports strings matching a catalog of credential patterns: cloud keys, version-control and package tokens, AI provider keys, payment keys, communication webhooks, SaaS API keys, JWTs, PEM private keys, and connection strings with embedded credentials.

```
$ mfi scan -root <extracted-tree>
```

```
3 finding(s)
[aws-access-key-id] /home/op/cap/shared_prefs/aws.xml:4 AKIA...(20 chars)
[jwt] /home/op/cap/databases/session.db:1 eyJh...(214 chars)
[generic-secret-assignment] /home/op/cap/files/config.json:12 "api...(37 chars)
```

Each finding reports the rule that matched, the file and line, and a **redacted fingerprint**: the first four characters plus the total length. The raw value is never printed by `mfi scan`.

Flags

Flag	Meaning
<code>-root</code>	Extracted tree to scan (required)
<code>-known</code>	File of known secrets to also search for, one per line
<code>-verify</code>	Live-verify findings against each service's API (opt-in, network)

The rule catalog

Forty-four built-in rules, grouped by what they protect:

Cloud and infrastructure: AWS access key ids, GCP API keys, GCP OAuth client secrets, GCP service-account key ids, DigitalOcean tokens, Databricks tokens, Doppler tokens, Terraform Cloud tokens.

Version control and packages: GitHub tokens (classic and fine-grained), GitLab personal access tokens, npm tokens, PyPI tokens, Postman API keys.

AI providers: OpenAI, Anthropic, Hugging Face.

Payments: Stripe secret and restricted keys, Square access tokens, Braintree access tokens.

Communication and email: Slack tokens, app tokens and webhooks, Discord bot tokens and webhooks, Telegram bot tokens, Twilio API keys, SendGrid, Mailgun, Mailchimp.

SaaS APIs: Shopify, Notion, Linear, Airtable, New Relic, Grafana.

Generic and structural: JWTs, PEM private key headers, MongoDB and SQL and Redis connection URIs with embedded credentials, HTTP basic-auth URLs, `key = "value"` style secret assignments, and `Bearer` tokens.

Every pattern is anchored on the issuer's published token format (the public prefix, character alphabet, and length), which keeps false positives low. Patterns compile under Go's RE2 engine, so no input can trigger catastrophic backtracking.

The generic rules

`generic-secret-assignment` and `bearer-token` are deliberately broader than the vendor-specific rules. They catch credentials for services with no distinctive token format, at the cost of matching some non-secrets (a config key literally named `password` holding a placeholder, for instance). Expect to triage these by hand. The vendor-specific rules are high-confidence; the generic pair are leads.

Known-secret lists

When you already hold a credential and want to know whether it appears in the app's data, put it in a file (one per line, blank lines and `#` comments ignored) and pass it in:

```
$ cat known.txt
# Credentials issued for this engagement
hunter2SuperSecretValue
sk_test_51H9xExampleKeyMaterial

$ mfi scan -root ./cap -known known.txt
```

Matches report under the rule id `known-secret`. Literal values are regex-quoted before use, so metacharacters in your secrets are matched literally and cannot corrupt the scanner's patterns.

Use this to answer questions like:

- Does the app persist the session token issued at login?
- Did the credential I typed into the login form end up in plaintext on disk?
- Is the API key from the app's build config also present at runtime?

What the scanner skips

Limit	Value	Rationale
Max file size	16 MB	Larger files are skipped entirely
Max line length	1 MB	A file with a longer line is abandoned
Max matches per rule per line	50	Prevents one pathological line dominating
Binary detection	First 512 bytes	A NUL byte in the sniff window marks the file binary and skips it

The binary skip matters for interpretation: **a secret embedded in a compiled binary, an image, or a compressed archive will not be found**. The scanner works on text. If you suspect a credential in a binary artifact, extract the strings yourself and scan that output, or use the Decode tab on a suspicious blob.

Live verification

`-verify` answers a different question from “is there a credential-shaped string here”: it answers **is this credential still live**.

```
$ mfi scan -root ./cap -verify
verifying findings against their services...
2 finding(s)
[github-token] /home/op/cap/files/config.json:8 ghp_...(40 chars) [active]
[stripe-secret-key] /home/op/cap/files/config.json:9 sk_l...(107 chars) [inactive]
```

Statuses: `active`, `inactive`, `unknown` (the check could not complete), and `unsupported` (the rule has no verifier).

This sends the matched secret to the service that issued it. Read that sentence twice before using the flag on a client engagement. Every verifier calls a read-only “whoami”-style endpoint over HTTPS: nothing is created, modified, or deleted. But the credential does leave your workstation, and the request appears in the service’s audit log, attributable to your source IP and the time you ran it.

When to use it:

- The finding count is large and you need to prioritise triage.
- You need to demonstrate real impact (“this key is live”) rather than theoretical exposure.
- The client has authorised outbound verification in scope.

When not to use it:

- The engagement forbids outbound traffic to third parties.
- The credentials belong to a third party that has not consented.
- You are working on an air-gapped or otherwise contained host.

Identical secrets are verified once, no matter how many files they appear in. Verification runs with a bounded concurrency and a short per-request timeout, so a hanging provider does not stall the scan indefinitely.

Rules with verifiers cover the major providers (GitHub, GitLab, AWS, OpenAI, Anthropic, Stripe, Slack, Postman, Notion, Airtable); everything else reports `unsupported`.

Interpreting findings

A finding is a lead, not a conclusion. Triage in this order:

1. **Is it real?** Open the file and look at the context. `mfi render -file <path>` shows it in a readable form. Placeholders, example keys from documentation, and test fixtures all match real patterns.
2. **Is it reachable?** A credential in an app's private data directory is only exposed to an attacker who can already read that directory: another app with root, a device thief with an unlocked handset, a malicious backup consumer. Say which threat model applies.
3. **Is it live?** Either `-verify`, or manual validation against the service if outbound verification is out of scope.
4. **What does it grant?** A read-only analytics key and a production payments key are the same shape and wildly different findings.

Where the raw values are

`mfi scan` never prints raw secrets. To retrieve them:

- **CLI:** `mfi report -root <tree> -show-secrets` includes raw values in the report. Do not share that output.
- **GUI:** click a redacted value in the Scan tab to reveal it, or use the row's Copy action.

In the GUI

The Scan tab adds:

- A **progress indicator** with file counts, and a cancel button.
- **Sortable columns** by rule, path, line, and verification status.
- **Click-to-reveal** on the redacted match column.

- Per-row actions: **Render** (open the file with the secret highlighted), **Decode** (send the value to the Decode tab), and **Copy**.
- A **Live verify** checkbox, gated behind a confirmation dialog that states plainly what the verification does.

Next

- Diffing: what changed between two captures.
- Reporting: turn findings into a shareable artifact.

Diffing captures

Diffing compares two extracted trees and reports what changed. It is the highest-signal technique in the toolkit: instead of reading an app's entire data directory hoping something stands out, you perform an action on the device and see exactly which files the action touched.

```
$ mfi diff -a <first-root> -b <second-root>
```

```
7 change(s) between /home/op/cap-loggedout and /home/op/cap-loggedin
added    databases/session.db
modified shared_prefs/auth.xml (json: 3 field(s) changed)
modified databases/app.db (sqlite: 2 table(s) changed, 14 row(s) differ)
removed  files/onboarding.flag
```

Change kinds

Kind	Meaning
added	Present only in the second tree
removed	Present only in the first tree
modified	Present in both, contents differ

The comparison is by path relative to each root, so the two trees must be captures of the same app for the output to mean anything.

Structural diffing

For file-level `modified` entries, MobFI does not stop at “these bytes differ”. Three structural differs produce a semantic description:

SQLite: compares table by table and reports how many tables changed and how many rows differ. A database whose file bytes changed but whose rows did not (a vacuum, a journal checkpoint, a timestamp in the header) reports `sqlite: no row differences (metadata only)`, which saves you opening it.

JSON: compares documents structurally and counts changed fields rather than changed lines. Key reordering and whitespace changes produce `json: no field differences`.

Property lists: handles binary and XML plists, including comparing a binary plist against an XML one, since the underlying data model is the same.

When no structural differ recognises the file, or a structural diff fails, MobFI falls back to a byte-level description (size and hash change), with the failure reason appended when one occurred.

This is why the diff is worth running even on captures where you already know something changed: the structural detail tells you *what* changed, not just *that* it did.

The canonical workflow

The reason to diff is to attribute data to an action.

```
# 1. Baseline: app installed, never logged in
$ mfi extract -device <id> -app com.example.target -out ./cap-01-fresh

# 2. Perform exactly one action on the device: log in

# 3. Capture again
$ mfi extract -device <id> -app com.example.target -out ./cap-02-loggedin

# 4. What did logging in write?
$ mfi diff -a ./cap-01-fresh -b ./cap-02-loggedin
```

Every file in that diff is part of the app's authenticated state. That is a far better place to hunt for session tokens than the whole tree.

Actions worth diffing around:

Action	What you are looking for
Log in	Session tokens, refresh tokens, credential caches
Log out	Whether the above are actually cleared
Enable biometrics	Key material, keychain or keystore references
Save a payment method	Card data, tokenised references
Receive a push notification	Notification payload persistence
Go offline then online	Sync state, cached responses
Change a privacy setting	Whether the setting is enforced locally or only in the UI

The logout case is the most productive finding generator in mobile testing: apps very often write a token on login and fail to remove it on logout.

Sequencing captures

Number your capture directories in the order you took them. The alternative is discovering three days later that you cannot tell which of `cap-a` and `cap-b` came first:

```
captures/
  2026-07-31-target/
    cap-01-fresh/
    cap-02-loggedin/
    cap-03-loggedout/
    cap-04-biometrics-on/
```

Then diff whichever pair answers your question:

```
$ mfi diff -a cap-02-loggedin -b cap-03-loggedout # what did logout clear?
$ mfi diff -a cap-01-fresh    -b cap-03-loggedout # what survived the round trip?
```

The second comparison is the interesting one. Anything present in `cap-03` that was absent in `cap-01` outlived a full login/logout cycle.

Noise

Every diff of a real app contains noise. Common sources:

- **Log files and caches.** Timestamps, request logs, image caches.
- **SQLite WAL and journal files.** `-wal` and `-shm` siblings change constantly. The structural differ helps here: the main database often reports “metadata only” while the WAL churns.
- **Analytics queues.** Batched events accumulate between captures.
- **Crash reporter state.** Session ids that rotate on every launch.

None of this is a bug in the diff. Filter it in your head, or with `grep -v`:

```
$ mfi diff -a ./cap-01 -b ./cap-02 | grep -vE 'cache/|\.log$|-wal$|-shm$'
```

To reduce noise at the source, minimise the time and activity between captures: take the second capture immediately after the action, with the app backgrounded rather than actively running.

Combining with a scan

The report command runs both in one pass and puts them in one artifact:

```
$ mfi report -root ./cap-02-loggedin \  
            -a ./cap-01-fresh -b ./cap-02-loggedin \  
            -out ./report.html
```

Findings from the scan and changes from the diff land in the same report, which is what you want for a writeup: “logging in wrote `session.db`, and `session.db` contains a live JWT”.

In the GUI

The Diff tab takes two root directories and shows the change list with the same structural details. Rows carry a **Compare** action as the primary button, which opens both versions of the file side by side in the render pane, so you can read the actual difference rather than the summary of it.

Next

- Database viewing: open the SQLite files a diff pointed you at.
- Reporting: aggregate scan and diff into a shareable artifact.

Database viewing

Mobile apps keep their interesting state in SQLite. Session tokens, cached API responses, message history, analytics queues, and credential caches all tend to land in a `.db` file under the app's data directory. MobFI opens them read-only so you can list tables and dump rows without touching the evidence.

```
# List tables
$ mfi db -file <path-to.db>

# Dump a table
$ mfi db -file <path-to.db> -table <table> [-limit N]
```

Flag	Default	Meaning
<code>-file</code>	(required)	SQLite database file
<code>-table</code>	(none)	Table to dump. Omit to list tables
<code>-limit</code>	100	Maximum rows to read

Listing and dumping

```
$ mfi db -file ./cap/databases/app.db
4 table(s):
  sessions
  messages
  cached_responses
  android_metadata

$ mfi db -file ./cap/databases/app.db -table sessions -limit 5
id  user_id  token  created_at
1   4471    eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...  1722384000
(1 row(s))
```

User tables are listed; SQLite's internal `sqlite_*` tables are omitted.

The table name you pass is validated against the database's actual table list before use, and quoted defensively, so a table name cannot be used to inject SQL.

Finding the databases in a capture

```
$ find ./cap -name '*.db' -o -name '*.sqlite' -o -name '*.sqlite3'
```

Android apps conventionally use `databases/`; iOS apps scatter them more widely, often under `Library/` or `Documents/`. File extension is a weak signal on iOS: MobFI's renderer

detects SQLite by the file header, so if you are unsure whether a file is a database, run `mfi render -file <path>` and see what it reports.

Read-only and evidence integrity

Evidence: MobFI opens databases read-only. The main database file is never written.

Be aware of one caveat on this version: a database carrying a hot write-ahead log (a `-wal` sibling) cannot be opened in SQLite's strictest immutable mode, because the WAL must be read to see committed rows. In that case MobFI falls back to a plain read-only open, and SQLite may create or update `-wal` and `-shm` sidecar files next to the database.

The practical consequence for chain of custody: **hash the capture directory before browsing databases**, not after. If sidecar mutation is unacceptable for your evidence handling, copy the database (and its `-wal` / `-shm` siblings) to a scratch directory and point MobFI at the copy.

```
$ mkdir -p /tmp/scratch
$ cp ./cap/databases/app.db* /tmp/scratch/
$ mfi db -file /tmp/scratch/app.db -table sessions
```

Cell rendering

Values are rendered for reading, not for byte-exact export:

- `NULL` prints as `NULL`.
- Text blobs print as text.
- Binary blobs print as `<blob N bytes>` rather than dumping raw bytes into your terminal.

When a blob matters, extract it and inspect it separately: the Decode tab handles base64 and hex, and `mfi render` will hex-dump anything it does not recognise.

What to look for

A checklist that covers most mobile findings:

Table or column pattern	Why it matters
session , token , auth , credential	Session material persisted to disk
user , account , profile	PII, and the identity bound to the session
cache , response , http	Cached API responses, often containing the above
message , chat , conversation	Message content, often unencrypted at rest
analytics , event , queue	What the app reports about the user
key , secret , cert	Key material the app manages itself
Any column holding eyJ...	A JWT. Decode it

Cross-reference with the scan: a finding at `databases/app.db:1` tells you the secret is in a database, and this is how you see its row context.

Decoding what you find

Values in databases are frequently encoded. Pull the value and hand it to the decoder:

```
$ mfi decode 'eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0NDcxIn0.abc'
Base64:  {"alg":"HS256"}
Hex:     no hex digits
URL:     no percent-encoding found
```

For JWTs, decoding the first segment gives you the algorithm, and the second gives you the claims (including expiry, which tells you whether the token you found is still valid).

In the GUI

The Database tab lists tables as clickable chips, dumps rows into a sortable and resizable table, and keeps the header row frozen while the body scrolls. The row limit is adjustable per query.

The Render tab recognises SQLite files and offers an **Open in Database** button, so a database you spot while browsing the tree is one click from being queried.

Next

- Rendering and decoding: read the non-database files.
- Keys: recover credential-store material.

Rendering and decoding

An extracted tree is full of formats that are unreadable in a plain text editor: binary property lists, SQLite files, minified JSON, opaque blobs. Rendering makes them legible; decoding turns encoded strings back into their contents.

Rendering a file

```
$ mfi render -file <path>
```

```
# application/x-plist

{
  "SessionToken": "eyJhbGciOiJIUzI1NiJ9...",
  "UserID": 4471,
  "LastSync": 2026-07-31T09:14:22Z
}
```

The output begins with the detected MIME type, then the rendered content.

Renderer selection

MobFI picks a renderer by **content**, not by file extension, trying each in priority order and using the first that recognises the file:

Order	Renderer	Detects	Output
1	SQLite	SQLite format 3 header	Table summary
2	JSON	Parses as JSON	Pretty-printed, indented
3	Property list	Binary bplist00 magic or XML plist	Decoded structure
4	XML	Parses as XML	Reindented
5	Text	Valid UTF-8, no control bytes	As-is
6	Hex dump	(catch-all)	Classic offset/hex/ASCII dump

Content detection matters on iOS especially, where a file named `.plist` may be XML or binary, and a file with no extension at all may be either. You do not need to know which; render it and see.

Rendering caps input at **1 MB**. Files larger than that are truncated with a marker at the cut point. To read past the cap, use your own tooling on the extracted file: the capture on disk is the complete file, only the rendered view is bounded.

Practical uses

Read a binary plist without converting it first:

```
$ mfi render -file ./cap/Library/Preferences/com.example.target.plist
```

Summarise a database you have not decided to query yet:

```
$ mfi render -file ./cap/databases/app.db
```

See what an extensionless blob actually is:

```
$ mfi render -file ./cap/files/blob_0041
```

If it hex-dumps, look at the first bytes: `PK` is a zip, `\x89PNG` is an image, `bplist00` a binary plist that failed to parse (possibly truncated or corrupt).

Decoding strings

```
$ mfi decode <string>
$ mfi decode -input <string>
$ echo <string> | mfi decode
```

All three forms work; use whichever fits your pipeline. The decoder runs every decoder over the input and reports each result:

```
$ mfi decode 'SGVsbG8sIG9wZXJhdG9y'
Base64: Hello, operator
Hex:    odd number of hex digits
URL:    no percent-encoding found
```

The decoders

Base64 tries standard and URL-safe alphabets, padded and raw. An input that decodes under any of the four is reported.

Hex ignores whitespace, `0x` prefixes, and `:` `/` `-` `,` separators, so `48 65`, `0x4865`, and `48:65` all decode.

URL percent-decodes, form-style (so `+` becomes a space). It only applies when the input actually contains a `%`, to avoid reporting every plain string as a successful no-op decode.

Binary results

When a decode produces bytes that are not printable text, MobFI reports it as binary and shows a hex view instead of dumping control characters into your terminal:

```
$ mfi decode 'H4sIAAAAAAAAA/NIzcnJVYjPL8pJAQBWsRdKCwAAAA=='
Base64: (binary) 1f 8b 08 00 00 00 00 00 03 f3 48 cd c9 c9 ...
```

`1f 8b` is a gzip header: that value is a compressed payload, not a credential.

The hex view is capped at 4096 bytes so a large decode does not flood the output.

Chaining decodes

Layered encoding is common in mobile apps. Decode iteratively:

```
$ mfi decode 'JTdCJTlYdG9rZW4lMjILM0ElMjJhYmMlMjILN0Q='
Base64: %7B%22token%22%3A%22abc%22%7D

$ mfi decode '%7B%22token%22%3A%22abc%22%7D'
URL: {"token": "abc"}
```

Base64 wrapping URL-encoding wrapping JSON. Two passes and you have the plaintext.

In the GUI

The **Render** tab is a file browser plus a viewer:

- Navigate the extracted tree, click a file to render it.
- Syntax highlighting for code and structured formats.
- Images render inline; PDFs render in an embedded viewer.
- A **hex** toggle forces a hex dump of any file.
- A **wrap** toggle for long lines.
- **Open externally** hands the file to your OS default handler.

Do not use Open externally on files from an untrusted device. It invokes your operating system's handler, which will execute an executable, open a shortcut's target, or run a script. The extracted tree is attacker-controlled data. If you must open something externally, copy it out to a scratch directory first and inspect the name and type deliberately.

The **Decode** tab takes a pasted string and shows every decoder's result at once, with a copy button per result. The Scan tab's per-row **Decode** action sends a finding straight there, which is the fastest route from "the scanner found a JWT" to "here are its claims".

Next

- Keys: recover keychain and keystore material.
- Reporting: aggregate what you found.

Keychain and Keystore

Apps that handle credentials properly do not leave them in `shared_prefs` or a plist. They put them in the platform credential store: the iOS Keychain or the Android Keystore. This chapter covers what MobFI can recover from each, and the hard platform limits that decide what is recoverable at all.

```
$ mfi keys -platform ios -device <udid> -state jailbroken
$ mfi keys -platform ios -backup <dir> -password <pw>
$ mfi keys -platform android -device <serial> -state rooted
```

Flag	Meaning
<code>-platform</code>	<code>ios</code> or <code>android</code> . Inferred as <code>ios</code> when <code>-backup</code> is given
<code>-device</code>	UDID (iOS) or serial (Android)
<code>-state</code>	<code>jailbroken</code> (iOS) or <code>rooted</code> (Android)
<code>-backup</code>	iOS: path to an encrypted backup directory
<code>-password</code>	iOS: backup password (or set <code>MFI_BACKUP_PASSWORD</code>)
<code>-reveal</code>	Include raw secret values. Default is redacted

The hard limits

Read this before planning work around key recovery.

iOS Secure Enclave keys are non-exportable. Not “hard to extract”: the private key material never leaves the Enclave, by design. MobFI can tell you a key exists and what it is used for. It can never dump it.

Android hardware-backed keys (TEE/StrongBox) are non-exportable. Same principle. Even with root, you get an inventory of which keys exist, not the key material.

iOS `ThisDeviceOnly` keychain items are excluded from backups by iOS itself, so the backup route cannot recover them regardless of password.

If your goal is the plaintext key an app *uses* rather than the stored key material, storage dumping is the wrong technique. Hook the app at runtime with Frida or objection and capture the key at the point of use.

iOS: the two routes

MobFI picks the best available method and degrades gracefully. When no method can run, it returns an explanation rather than an error, so you always get actionable guidance.

Encrypted backup (works on stock devices)

The only route that works on a normal, non-jailbroken iPhone.

1. **Enable backup encryption** on the device (Finder/iTunes, or via the device's own settings) and set a backup password. Backup encryption is what causes the keychain to be included at all: an unencrypted backup contains no keychain.
2. **Produce a backup.** MobFI's extraction `backup` scope does this (see the Extraction chapter), or use `idevicebackup2` directly.
3. **Point mfi keys at the backup directory** with the password.

```
$ export MFI_BACKUP_PASSWORD='the-backup-password'
$ mfi keys -backup ~/captures/backup-dir
```

The pipeline: parse `Manifest.plist`, unlock the backup keybag with the password, decrypt `Manifest.db`, locate and decrypt the keychain file, then parse its items. Every stage degrades and reports rather than failing hard.

If the backup is not encrypted, MobFI says so plainly:

```
this backup is not encrypted; the keychain is only present in
encrypted backups (enable backup encryption and re-run)
```

Jailbroken device over USB

On a jailbroken device running `sshd` with `keychain_dumper` installed, MobFI dumps the keychain directly over a USB port forward.

```
$ mfi keys -platform ios -device <udid> -state jailbroken
```

You must tell MobFI the device is jailbroken with `-state jailbroken`. It does not probe for a jailbreak.

If both a backup and `-state jailbroken` are available, the backup route is tried first, and the jailbreak route is used as a fallback if backup decryption fails.

Android: rooted device

```
$ mfi keys -platform android -device <serial> -state rooted
```

The Android Keystore's blobs live under `/data/misc/keystore`, owned by root, so root is mandatory. MobFI reads the keystore2 database (Android 12 and later) and reports the key entries with their security levels: software, TEE, or StrongBox.

Interpreting the security level is the point of the exercise:

Level	Meaning	Recoverable?
Software	Key material in software, protected by the keystore	Potentially
TEE	Key lives in the Trusted Execution Environment	Inventory only
StrongBox	Key lives in a dedicated secure element	Inventory only

An app storing a signing key at software security level on a rooted device is a finding. The same app on a device where the key is StrongBox-backed is not.

Reading the output

```
Method: encrypted backup
  note: Unlocked the backup keybag.
  note: Recovered 24 keychain item(s).
Items: 24
  [Generic Password] keychain service="com.example.target" account="session"
accessible=AfterFirstUnlock value=[hidden text, 213 bytes]
  [Internet Password] keychain service="api.example.com" account="user@example.com"
accessible=WhenUnlocked value=[hidden text, 32 bytes]
  [Key] keychain label="com.example.target.signing" accessible=AfterFirstUnlockThisDeviceOnly
value=[hidden binary, 256 bytes]
  limitation: Items protected 'ThisDeviceOnly' and Secure Enclave keys are excluded from
backups by iOS and cannot be recovered here.
```

Field	Meaning
Method	Which route ran. <code>unavailable</code> means none could
Degraded	Marked when the method could not run at all
note	What happened during the dump
limitation	What this method structurally cannot reach
accessible	The item's protection class

Protection classes

The `accessible` field is often the finding, independent of the value:

Class	Available when	Risk signal
<code>Always</code>	Always, even locked	Deprecated by Apple, weakest
<code>AfterFirstUnlock</code>	After first unlock since boot	Readable by background access post-boot
<code>WhenUnlocked</code>	Only while unlocked	Reasonable default
<code>*ThisDeviceOnly</code>	As above, never leaves the device	Strongest, excluded from backups

A session token stored `Always` or `AfterFirstUnlock` without the `ThisDeviceOnly` suffix is reachable in more circumstances (and survives into backups) than most developers intend.

Revealing values

Values are redacted by default, shown as `[hidden text, N bytes]` or `[hidden binary, N bytes]`. Add `-reveal` to include them:

```
$ mfi keys -backup ~/captures/backup-dir -reveal
```

Binary values print as a hex preview, capped at 512 bytes.

Treat revealed output as raw credential material. Do not paste it into a ticket, do not commit it, and do not leave it in shell history. On a shared host, prefer redirecting to a file with tight permissions over letting it scroll in a terminal that may be logged.

In the GUI, the Keys tab has a **Reveal secrets** checkbox, gated behind a confirmation dialog, and the setting persists across dumps so you do not have to re-tick it every time.

When nothing is recoverable

The degraded result tells you exactly what to change:

iOS, non-jailbroken, no backup: make an encrypted backup and re-run against it.

Android, not rooted: no route exists. The keystore is root-only. Consider runtime hooking instead.

Neither is a MobFI failure; both are the platform working as designed.

Next

- Reporting: aggregate findings into a shareable artifact.
- Console: interactive shell access for follow-up.

Reporting

The report command aggregates a secrets scan and a diff into one artifact, in text, JSON, or HTML.

```
$ mfi report -root <tree> # scan only
$ mfi report -a <root-a> -b <root-b> # diff only
$ mfi report -root <tree> -a <root-a> -b <root-b> # both
```

Flag	Meaning
-root	Tree to scan for secrets
-known	Known-secrets file to add to the scan
-a , -b	Two roots to diff
-out	Also write to this file. Format follows the extension
-show-secrets	Include raw, unredacted secrets
-verify	Live-verify findings (opt-in, network)

At least `-root` , or both `-a` and `-b` , is required.

Output formats

The format follows the extension of `-out` :

Extension	Format
.html , .htm	Self-contained HTML, inline CSS, no external assets
.txt	Plain text, same as stdout
anything else	JSON

```
$ mfi report -root ./cap -out ./report.html # HTML
$ mfi report -root ./cap -out ./report.json # JSON
$ mfi report -root ./cap -out ./report.txt # text
```

The text summary always prints to stdout as well, so you see the result immediately and get the file for later.

Text

```
MobFI report - 2026-07-31T14:22:08Z

Secrets: 3 finding(s)
  aws-access-key-id          1
  jwt                        1
  generic-secret-assignment  1
  - [aws-access-key-id] /home/op/cap/shared_prefs/aws.xml:4 AKIA...(20 chars)
  - [jwt] /home/op/cap/databases/session.db:1 eyJh...(214 chars)

Diff: 7 change(s) (added 3, removed 1, modified 3)
  added   databases/session.db
  modified shared_prefs/auth.xml (json: 3 field(s) changed)
```

Good for terminal review and for pasting into notes.

JSON

Machine-readable, for pipelines and for importing into a reporting platform. Contains the findings array (rule id, path, line, redacted match, verification status), the diff result, and generation metadata.

```
$ mfi report -root ./cap -out ./report.json
$ jq '.findings[] | select(.verified=="active")' ./report.json
```

HTML

A single self-contained file with inline CSS and no external assets, so it renders identically offline and can be attached to a ticket or emailed. Values are HTML-escaped, so attacker-controlled paths and matches from the extracted data are safe to embed.

Redaction

By default, reports contain only redacted fingerprints (first four characters plus length). That is the shareable form.

`-show-secrets` retains the raw values in every output format:

```
$ mfi report -root ./cap -show-secrets -out ./report-raw.json
```

The report marks itself:

```
Secrets: 3 finding(s) [UNREDACTED - contains raw secrets]
```

Use it for authorized local analysis. Do not share the output, do not attach it to a ticket, and delete it when the analysis is done. If you need to hand a client the *evidence* that a credential was exposed, the redacted fingerprint plus the file path is normally sufficient, and the raw value can be provided separately through whatever secure channel the engagement uses.

Paths in reports

Findings record the path as walked, so if you scanned an absolute path, the report contains absolute paths:

```
{"rule_id":"jwt","path":"/home/christoff/engagements/acme-2026/cap/databases/session.db","line":1}
```

That leaks your username, your directory layout, and possibly the client's codename to anyone who reads the report.

Two mitigations:

Scan with a relative path, from inside the parent directory:

```
$ cd ~/engagements/acme-2026
$ mfi report -root ./cap -out ./report.json # paths are ./cap/...
```

Or post-process before sharing:

```
$ jq '(.findings[].path) |= sub("^/home/[^/]+/engagements/[^/]+/"; "")' \
  report.json > report-clean.json
```

Check the report before it leaves your machine. This is the most common accidental disclosure in tooling output.

Verification in reports

`-verify` adds live verification (see the Scanning chapter for what that sends and where):

```
$ mfi report -root ./cap -verify -out ./report.html
```

Verified findings carry `active`, `inactive`, `unknown`, or `unsupported`, which makes the report far more actionable: a client reading “3 credentials found, 1 confirmed live” prioritises differently from “3 credentials found”.

A complete engagement flow

```
$ cd ~/engagements/acme-2026

# Capture baseline, act on the device, capture again
$ mfi extract -device <id> -app com.acme.mobile -out ./cap-01-fresh
$ mfi extract -device <id> -app com.acme.mobile -out ./cap-02-loggedin

# One report covering both the scan and what login changed
$ mfi report -root ./cap-02-loggedin \
  -a ./cap-01-fresh -b ./cap-02-loggedin \
  -known ./known-secrets.txt \
  -out ./report.html

# Keep a JSON copy for the reporting platform
$ mfi report -root ./cap-02-loggedin \
  -a ./cap-01-fresh -b ./cap-02-loggedin \
  -out ./report.json
```

Evidence: keep the captures. The report references file paths and line numbers that are only meaningful alongside the tree they came from. Archive `cap-*` and the report together.

In the GUI

The Report tab builds the same artifact from the last scan and diff performed in the session, with an export button per format. The unredacted export is gated behind a confirmation dialog that states what the file will contain.

Next

- Console: interactive device shell for follow-up questions.
- Updating: keep MobFI current.

Console

The Console is an interactive terminal to the device, embedded in the desktop GUI. Use it when a capture raises a question that only the live device can answer: what does that directory look like right now, is that process running, what does the app write when you tap the button.

The Console is **GUI only**. From the CLI, run `adb shell` or `ssh` directly; the Console adds a terminal in the same window as your captures, plus session transcript logging.

Android

Select the device, open the Console tab, and start a session. MobFI runs `adb -s <serial> shell` in a pseudo-terminal, so interactive programs, job control, and colour output all behave normally.

Useful once you are in:

```
# What can the shell user actually read?
$ run-as com.example.target ls -la /data/data/com.example.target

# Is the app running, and as which uid?
$ ps -A | grep example

# What did the app just write?
$ run-as com.example.target ls -lt /data/data/com.example.target/files | head

# Live log for the target
$ logcat --pid=$(pidof com.example.target)
```

The `run-as` prefix is the same mechanism extraction uses: it works for debuggable apps without root. On a rooted device, `su` gives you the same reach for any app.

iOS

iOS Console requires a **jailbroken device running sshd**. Stock iOS has no shell to connect to.

Two transports:

USB (recommended): MobFI starts `iproxy` to forward a local port to the device's SSH port, then connects `ssh` through it. Select the device and leave the host field empty.

Network: supply the device's hostname or IP directly. Fill in the SSH host field.

The user defaults to `root` and the port to `22`; both are adjustable in the tab.

Useful once you are in:

```
# Find the app's container
$ find /var/mobile/Containers/Data/Application -maxdepth 2 -name '*.plist' 2>/dev/null

# What is installed
$ ls /var/containers/Bundle/Application/

# Watch the app's files change
$ ls -lt /var/mobile/Containers/Data/Application/<uuid>/Documents
```

OPSEC: SSH to a device on the network is visible to anyone watching that network segment, and the connection appears in the device's own logs. Over USB via `iproxy`, the traffic stays on the cable.

Host-key handling

On this version, the Console connects with `StrictHostKeyChecking=accept-new` and `UserKnownHostsFile=/dev/null`, meaning the host key is accepted and then forgotten. On a loopback USB forward that is reasonable: the trust boundary is the cable.

Over a **network** connection it means no host-key drift detection: an attacker on the network segment who can present a forged key will not trigger a warning. When connecting to a device over an untrusted network, prefer the USB transport, or SSH manually with a real known-hosts file:

```
$ ssh -o UserKnownHostsFile=~/.ssh/known_hosts_mobfi \
      -o StrictHostKeyChecking=accept-new root@<device-ip>
```

Session transcripts

Supply a log path when starting a session and MobFI writes a transcript of everything the session produced.

Evidence: a transcript is the record of what you ran on the device and what it returned. For any engagement where your actions on a device may be questioned later, turn it on. Store the transcript alongside the captures for that device.

The transcript captures session output, including anything you type that the device echoes back. Assume **credentials typed into the session end up in the file**, and handle it accordingly.

Practical patterns

Confirm an extraction gap. The extract reported skipped paths; check whether they are genuinely unreadable or whether the wrong mechanism was used:

```
$ run-as com.example.target ls -la /data/data/com.example.target/cache
```

Watch a file appear. Diffing tells you a file changed between captures; the Console tells you when it changes:

```
$ watch -n1 'run-as com.example.target ls -l /data/data/com.example.target/files'
```

Check the app's own view. The app may hold data in memory that never lands on disk. `dumpsys` and `logcat` surface some of it without touching storage.

Verify a permission. The Apps tab lists granted permissions; the Console proves which are effective:

```
$ dumpsys package com.example.target | grep -A20 'runtime permissions'
```

In the GUI

The Console tab supports multiple concurrent sessions, each in its own tab, with:

- A full xterm-compatible terminal (scrollback, colour, resize).
- Copy and paste, including right-click copy on selected text.
- Per-session transcript logging.
- A status line naming the transport in use, so you always know whether you are on USB or the network.

Next

- Updating: keep MobFI current.
- Troubleshooting: when a workflow does not behave.

Updating

MobFI checks for updates and can update itself in place, whether you installed a prebuilt binary or built from a git checkout.

Checking

```
$ mfi update
```

```
Current: v1.0.0
Latest:  v1.1.0

A newer release is available: v1.1.0
https://github.com/integrise/MobFI/releases/tag/v1.1.0
Update now:  mfi update -apply
```

The check reports two independent signals:

- Whether a **newer published release** exists than the running version. This works for any install, including prebuilt binaries.
- When running from inside a **git checkout**, how many commits the local branch is behind its upstream.

Checking changes nothing on disk.

Machine-readable form:

```
$ mfi update -json
```

Applying

```
$ mfi update -apply
```

What happens depends on how MobFI was installed:

Git checkout: `git pull --ff-only` from the public HTTPS URL, then a rebuild via the project's install script. HTTPS rather than the configured remote (often SSH) so an unattended update needs no SSH key or agent.

Prebuilt binary: downloads the release asset for your platform, verifies its SHA-256 against the published `SHA256SUMS.txt`, and atomically replaces the running executable.

On Unix the replacement is a same-filesystem rename, which is atomic even while the old binary runs. Windows cannot overwrite a running image, so the old binary is moved aside first and rolled back if the swap fails.

Re-run `mfi` afterwards to use the new version.

Automatic notices

One-shot subcommands print a one-line notice to stderr after running if an update is available, so a piped stdout stays clean:

```
Update available: MobFI v1.1.0 (you have v1.0.0) - run `mfi update` for details.
```

The notice is skipped for `update`, `version`, and `help`, where a network check would be noise.

The wizard goes further: when stdin is an interactive terminal, it offers to update at launch, applies it with live progress, and re-execs the freshly-built binary so the wizard continues on the new version. When stdin is piped, it prints the notice only.

The GUI checks in the background and shows a dismissable banner with an “Update now” button. On macOS and Linux the update runs in-process with progress in the window; on Windows the app closes, a detached worker performs the swap in its own console window, and the app relaunches automatically.

Disabling the check

```
$ export MFI_NO_UPDATE_CHECK=1
```

Set this when:

- Working on an **air-gapped or contained host** where outbound traffic is not permitted.
- The engagement forbids **any unattributed outbound connection** from the testing workstation.
- You need **byte-identical tooling** across a test series and do not want a mid-engagement version change.

The check is a request to the GitHub releases API. It carries no device data, no capture data, and no identifying information beyond what any HTTPS client sends.

Version pinning for an engagement

For work where reproducibility matters, pin the version at the start and record it:

```
$ mfi version
mfi v1.0.0 (abc1234, 2026-07-31)

$ export MFI_NO_UPDATE_CHECK=1
```

Put the full version string in your engagement notes. If a finding is later questioned, you can rebuild the exact tool that produced it.

Verifying an update

After updating, confirm the version and that the tools still resolve:

```
$ mfi version
$ mfi doctor
```

A git-checkout update rebuilds from source, so a failed rebuild leaves the previous binary in place; the error is reported and nothing is silently half-installed.

Manual update

If the self-updater cannot run (no network, restricted host, policy), update by hand:

```
# Git checkout
$ git pull --ff-only
$ make build

# Prebuilt binary: download the new release, verify, replace
$ shasum -a 256 -c SHA256SUMS.txt --ignore-missing
$ mv mfi_v1.1.0_darwin_arm64 ~/.local/bin/mfi
```

If `scripts/install.sh` created a **symlink** into `~/.local/bin` (the default), a `git pull` plus `make build` is picked up automatically with no further steps.

Next

Troubleshooting: when something does not behave.

Troubleshooting

Symptoms, causes, and fixes. Work top-down: the ordering roughly matches how often each cause turns out to be the real one.

No devices detected

1. The platform tool is missing.

```
$ mfi doctor
```

No `adb` means no Android devices, ever. No `idevice_id` means no iOS devices.

2. The platform tool cannot see it either. This separates a MobFI problem from an environment problem:

```
$ adb devices -l
$ idevice_id -l
$ xcrun simctl list devices booted
```

If the native tool does not list the device, fix that first. MobFI cannot see what its tools cannot see.

3. The screen is locked. Both platforms gate authorisation prompts behind an unlocked screen.

4. The cable is charge-only. Swap it.

5. The daemon is wedged.

```
$ adb kill-server && adb start-server
$ sudo systemctl restart usbmuxd      # Linux, iOS
```

6. Linux udev rules are missing for Android. Without them `adb` reports `no permissions`.

Device shows `unauthorized` (Android)

The USB debugging prompt was never accepted, or the host key was revoked.

1. Unlock the device.
2. Replug.
3. Accept the prompt, ticking “always allow from this computer”.

If no prompt appears: Developer Options, “Revoke USB debugging authorisations”, then replug.

Device shows `unpaired` (iOS)

The host is not trusted by the device.

1. Unlock the device.
2. Replug.
3. Tap **Trust**, enter the passcode.

If no prompt appears: Settings, General, Transfer or Reset iPhone, Reset, Reset Location & Privacy, then replug.

Extraction returns 0 files

The single most common issue. In order of likelihood:

Wrong bundle id. Copy it exactly from `mfi apps`. A near-miss (`com.example.app` vs `com.example.app.debug`) produces exactly this symptom.

App is not debuggable and the device is not rooted (Android). `run-as` refuses, `su` is unavailable, the shell user has no access. Check the Apps tab details panel for a `DEBUGGABLE` package flag. Options: use a debuggable build, root the device, or accept that private data is out of reach.

Wrong iOS scope. `container` only works for dev-signed and jailbroken. Try `-scope documents`, then `-scope backup`.

App is not installed on that device. Verify with `mfi apps`.

Extraction is very slow

Per-file fallback is in use. The device has no `tar`, so MobFI copies file by file. Nothing to configure; it is slower by nature.

The backup scope is running. A full device backup precedes reconstruction and can take tens of minutes on a device with media. This is expected.

A huge cache directory. Some apps cache hundreds of megabytes of images. The capture includes them.

`cannot read /data/data/<pkg>`

The full message names the two likely causes, and it is usually right:


```
check the package name is exactly right and the app is installed;
a non-debuggable app needs root (approve the su/superuser prompt
on the device)
```

Check the package name character by character, then watch the device screen during the extract: a superuser prompt waiting for approval will block the attempt silently from the host's side.

iOS container extraction denied

```
(full container access needs a dev-signed/debug app or a jailbreak;
try the 'documents' scope, or 'backup' scope to pull a production
app's data)
```

Follow the advice in the order given: `documents` is fast and often sufficient; `backup` is slow but reaches production apps.

Backup extraction fails or hangs

Not enough disk space. A full backup needs room for the entire device's backed-up content, not just the target app.

The device locked mid-backup. Keep the device unlocked and plugged in for the duration.

A backup password is set but not supplied. For keychain recovery, the password is required (see the Keys chapter). For file extraction, an encrypted backup still works, but decrypting the manifest requires the password.

idevicebackup2 is missing. It is optional in `mfi doctor` precisely because only this scope needs it. Install `libimobiledevice` in full.

Scan finds nothing

The data is binary. The scanner skips files whose first 512 bytes contain a NUL. Secrets inside compiled binaries, images, or compressed archives are not found. Extract strings yourself and scan that output.

Files exceed the 16 MB cap. Larger files are skipped entirely.

The extraction was empty or partial. Check the extract's file count first: scanning an empty tree finds nothing, correctly.

The app genuinely stores nothing in plaintext. This is a good result, and worth reporting as one. Confirm it by checking whether the app uses the platform credential store instead (see the Keys chapter).

Scan finds far too much

Generic rules (`generic-secret-assignment` , `bearer-token`) are broad by design. Filter to the high-confidence rules for a first pass:

```
$ mfi scan -root ./cap | grep -vE 'generic-secret-assignment|bearer-token'
```

Then triage the generics separately.

Verification reports `unknown`

The check could not complete. Causes: no network, the provider’s API is unreachable, a proxy is interfering, or the provider changed its endpoint. `unknown` is not evidence that the credential is dead.

Keys reports `unavailable`

Expected on stock devices. The limitations in the output name the route that would work:

- **iOS, non-jailbroken:** make an **encrypted** backup, then point `mfi keys -backup <dir> -password <pw>` at it.
- **Android, not rooted:** no route exists; the keystore is root-only.

Backup keychain decryption fails

The backup is not encrypted. Only encrypted backups contain the keychain. Enable backup encryption on the device and make a new one.

Wrong password. MobFI detects this as a keybag unlock failure and reports “wrong backup password?” rather than producing garbage.

Items are `ThisDeviceOnly`. Those are excluded from backups by iOS. Nothing recovers them from a backup, by design.

GUI will not start

Missing WebKit toolchain (Linux). The install script installs GTK3 and WebKit2GTK packages; if you built manually, install them.

Missing WebView2 (Windows). Installed by the install script via winget.

Blank window on Linux. A known WebKit issue with some GPU drivers. Try

```
WEBKIT_DISABLE_COMPOSITING_MODE=1 mfi-gui .
```

GUI shows no app icons

`aapt` is missing. Cosmetic only: extraction, scanning, and everything else work identically. Install the Android SDK build-tools if you want real icons and names.

Console will not connect (iOS)

The device is not jailbroken. Stock iOS has no SSH server.

sshd is not running on the jailbroken device, or listens on a non-default port. Set the port in the Console tab.

iproxy is missing. It is optional in `mfi doctor`; install `libusbmuxd` (macOS) or `libusbmuxd-tools` (Linux).

adb over TCP will not connect

Pairing port used as the connect port. They differ. Pair with the dialog's host:port, then connect with the port from the main Wireless debugging screen.

Values expired. They rotate every time the pairing dialog is reopened. Read and use them promptly.

Client isolation on the network. Common on guest and corporate Wi-Fi. Use a network without it, or USB.

A VPN captured the route. Check with `route -n get <ip>` (macOS) or `ip route get <ip>` (Linux).

Filing a bug

Include:

- `mfi version` output.
- `mfi doctor` output.
- The exact command and its full output.
- Device platform, OS version, and state (rooted, jailbroken, stock).
- Whether the underlying tool (`adb` , `idevice_id`) sees the device.

Never include raw captured data, raw secrets, or an unredacted report in a bug report.

CLI reference

Every command and every flag. Running `mfi` with no arguments starts the guided wizard; `mfi help` lists the commands; `mfi <command> -h` lists the flags for one command.

Flags use Go's standard flag package, so `-flag value`, `-flag=value`, and `--flag value` are all accepted.

Command summary

Command	Purpose
<code>wizard</code>	Guided, step-by-step workflow (the default)
<code>detect</code>	List reachable Android/iOS devices
<code>apps</code>	List installed apps on a device
<code>extract</code>	Copy an app's file tree to a local directory
<code>scan</code>	Scan an extracted tree for secrets
<code>diff</code>	Compare two extracted roots
<code>report</code>	Scan and/or diff, then summarise
<code>db</code>	Inspect a SQLite database read-only
<code>render</code>	Render a file in a readable form
<code>decode</code>	Decode a string (Base64, hex, URL)
<code>keys</code>	Dump iOS Keychain / Android Keystore
<code>doctor</code>	Check for external device tools
<code>update</code>	Check for (and apply) a newer MobFI
<code>version</code>	Print the MobFI version
<code>help</code>	Show usage

`mfi wizard`

Guided workflow: detect, pick an app, extract, scan and/or diff, report. No flags. Type `q` or Ctrl-D at any prompt to quit.

```
$ mfi
$ mfi wizard
```

Reports produced by the wizard are always redacted.

`mfi detect`

Lists every reachable device across all detectors. No flags.

```
$ mfi detect
```

Columns: `ID` , `NAME` , `PLATFORM` , `TRANSPORT` , `STATE` . The `ID` value is what you pass to `-device` elsewhere.

A failing detector does not suppress the others; its error is reported after the device list.

`mfi apps`

Lists installed applications.

Flag	Default	Meaning
<code>-device</code>	(required)	Device ID from <code>mfi detect</code>
<code>-all</code>	false	Include system apps

```
$ mfi apps -device emulator-5554
$ mfi apps -device emulator-5554 -all
```

Columns: `BUNDLE ID` , `NAME` , `VERSION` , `DATA PATH` , `INSTALL PATH` .

`mfi extract`

Mirrors an app's on-device tree to a local directory.

Flag	Default	Meaning
<code>-device</code>	(required)	Device ID
<code>-app</code>	(required)	Package name or bundle id
<code>-out</code>	(required)	Local destination directory
<code>-scope</code>	<code>container</code>	iOS only: <code>container</code> , <code>documents</code> , or <code>backup</code>

```
$ mfi extract -device <id> -app com.example.target -out ./cap
$ mfi extract -device <udid> -app com.example.target -out ./cap -scope documents
$ mfi extract -device <udid> -app com.example.target -out ./cap -scope backup
```

Progress goes to stderr; the summary goes to stdout. Skipped paths are listed with their reasons.

`mfi scan`

Scans an extracted tree for secrets.

Flag	Default	Meaning
<code>-root</code>	(required)	Extracted tree to scan
<code>-known</code>	(none)	File of known secrets, one per line
<code>-verify</code>	false	Live-verify findings against each service's API

```
$ mfi scan -root ./cap
$ mfi scan -root ./cap -known ./known-secrets.txt
$ mfi scan -root ./cap -verify
```

Output is one line per finding: rule id, path, line, redacted fingerprint, and (with `-verify`) status. Raw secrets are never printed by this command.

`-verify` sends the matched secret to the issuing service. See the Scanning chapter before using it on an engagement.

`mfi diff`

Compares two extracted roots.

Flag	Default	Meaning
<code>-a</code>	(required)	First root
<code>-b</code>	(required)	Second root

```
$ mfi diff -a ./cap-01-fresh -b ./cap-02-loggedin
```

Each change reports `added`, `removed`, or `modified`, plus a structural detail for SQLite, JSON, and property-list files.

`mfi report`

Aggregates a scan and/or a diff into a report.

Flag	Default	Meaning
<code>-root</code>	(none)	Tree to scan
<code>-known</code>	(none)	Known-secrets file for the scan
<code>-a</code>	(none)	First root to diff
<code>-b</code>	(none)	Second root to diff
<code>-out</code>	(none)	Also write to this file, format by extension
<code>-show-secrets</code>	false	Include raw, unredacted secrets
<code>-verify</code>	false	Live-verify findings

Requires `-root`, or both `-a` and `-b`, or all three.

```
$ mfi report -root ./cap
$ mfi report -a ./cap-01 -b ./cap-02
$ mfi report -root ./cap -a ./cap-01 -b ./cap-02 -out ./report.html
$ mfi report -root ./cap -show-secrets -out ./report-raw.json
```

Format follows the `-out` extension: `.html` / `.htm` for HTML, `.txt` for text, anything else for JSON. The text summary always prints to stdout.

`mfi db`

Inspects a SQLite database read-only.

Flag	Default	Meaning
<code>-file</code>	(required)	Database file
<code>-table</code>	(none)	Table to dump. Omit to list tables
<code>-limit</code>	100	Maximum rows

```
$ mfi db -file ./cap/databases/app.db
$ mfi db -file ./cap/databases/app.db -table sessions
$ mfi db -file ./cap/databases/app.db -table sessions -limit 1000
```

`mfi render`

Renders a file in a human-readable form.

Flag	Default	Meaning
<code>-file</code>	(required)	File to render

```
$ mfi render -file ./cap/Library/Preferences/com.example.plist
```

Renderer chosen by content: SQLite, JSON, property list, XML, text, hex dump. Input is capped at 1 MB with a truncation marker.

`mfi decode`

Decodes a string as Base64, hex, and URL percent-encoding.

Flag	Default	Meaning
<code>-input</code>	(none)	String to decode

The input may also be a positional argument or piped on stdin:

```
$ mfi decode 'SGVsbG8='
$ mfi decode -input 'SGVsbG8='
$ echo 'SGVsbG8=' | mfi decode
```

Every decoder reports its result or why it did not apply. Binary results are shown as hex, capped at 4096 bytes.

mfi keys

Dumps the platform credential store.

Flag	Default	Meaning
<code>-platform</code>	(required unless <code>-backup</code>)	<code>ios</code> or <code>android</code>
<code>-device</code>	(none)	UDID (iOS) or serial (Android)
<code>-state</code>	(none)	<code>jailbroken</code> (iOS) or <code>rooted</code> (Android)
<code>-backup</code>	(none)	iOS: path to an encrypted backup directory
<code>-password</code>	(none)	iOS: backup password
<code>-reveal</code>	false	Include raw secret values

`-platform` is inferred as `ios` when `-backup` is given. `-password` falls back to `MFI_BACKUP_PASSWORD` .

```
$ mfi keys -platform android -device <serial> -state rooted
$ mfi keys -platform ios -device <udid> -state jailbroken
$ MFI_BACKUP_PASSWORD='pw' mfi keys -backup ./backup-dir
$ mfi keys -backup ./backup-dir -reveal
```

Prefer `MFI_BACKUP_PASSWORD` over `-password` on shared hosts: arguments are visible in the process table.

mfi doctor

Reports which external tools are installed.

Flag	Default	Meaning
<code>-json</code>	false	Machine-readable output

```
$ mfi doctor
$ mfi doctor -json
```

Status is `ok` , `MISSING` (a core tool), or `optional` .

mfi update

Checks for a newer MobFI, and optionally applies it.

Flag	Default	Meaning
<code>-json</code>	false	Machine-readable output
<code>-apply</code>	false	Perform the update

```
$ mfi update
$ mfi update -json
$ mfi update -apply
```

Set `MFI_NO_UPDATE_CHECK` to any value to disable automatic checks everywhere.

`mfi version`

```
$ mfi version
$ mfi --version
$ mfi -v
```

Prints the version, git commit, and build date.

`mfi help`

```
$ mfi help
$ mfi -h
$ mfi --help
```

Environment variables

Variable	Effect
<code>MFI_NO_UPDATE_CHECK</code>	Any value disables update checks
<code>MFI_BACKUP_PASSWORD</code>	iOS backup password for <code>mfi keys</code>
<code>MFI_UPDATED</code>	Set internally after a self-update re-exec

Exit codes

`0` on success, `1` on error with the message on stderr prefixed `mfi:` . Scripts should test the exit code rather than parsing output.

```
$ if ! mfi extract -device "$ID" -app "$PKG" -out "$OUT"; then
    echo "extraction failed" >&2
    exit 1
fi
```

Appendix

Repository layout

```

cmd/
  mfi/           CLI frontend
  mfi-gui/       Wails desktop frontend (Go + frontend/dist)
internal/
  app/           Framework-agnostic core; both frontends drive this
  backup/        iOS backup extraction and reconstruction
  dbview/        Read-only SQLite access
  decode/        Base64 / hex / URL decoders
  device/        Device models and detectors (adb, libimobiledevice, simctl)
  diff/          Tree diff plus SQLite / JSON / plist structural differs
  doctor/        External-tool detection and install hints
  extract/       Tree mirroring and path-safety guards
  keystore/      iOS Keychain and Android Keystore recovery
  plist/         Binary and XML property-list decoding
  render/        Per-format renderers with a hex-dump fallback
  report/        Text / JSON / HTML reporting
  secrets/       Rule catalog, scanner, live verification
  selfupdate/    Update check and in-place apply
  sysproc/       Process spawning (hides console windows on Windows)
  transport/     adb and AFC transports
  version/       Version constants
docs/
  handbook/      This handbook's chapter sources
  handbook.md    Generated: single-file handbook
  handbook.pdf   Generated: print artifact
scripts/
  build-handbook.sh Handbook generator
  install.sh      macOS / Linux installer
  install.ps1     Windows installer

```

All logic lives in `internal/`. The frontends are thin: if a behaviour differs between the CLI and the GUI, it is a frontend presentation difference, not a capability difference.

On-device paths worth knowing

Android

Path	Contents
/data/data/<pkg>/	App private data root (extraction target)
/data/data/<pkg>/shared_prefs/	XML preference files. Frequent source of plaintext tokens
/data/data/<pkg>/databases/	SQLite databases
/data/data/<pkg>/files/	Arbitrary app files
/data/data/<pkg>/cache/	Caches. Noisy in diffs
/data/app/<pkg>/	Installed APK
/data/misc/keystore/	Keystore blobs (root only)
/sdcard/Android/data/<pkg>/	External app storage, world-readable historically

iOS

Path	Contents
Documents/	User documents. Exposed when file sharing is enabled
Library/Preferences/	Property lists. Frequent source of plaintext tokens
Library/Caches/	Caches. Noisy in diffs
Library/Application Support/	App data, often SQLite
tmp/	Temporary files

Within a backup, data is organised by **domain** (`AppDomain-<bundle-id>` , `AppDomainGroup-*` , `KeychainDomain`) rather than by directory tree. MobFI's backup scope reconstructs the target's domains into a normal directory layout.

Glossary

AFC - Apple File Conduit. The protocol libimobiledevice uses to read files from an iOS device. "House arrest" is the AFC variant scoped to a single app's container.

adb - Android Debug Bridge. The tool MobFI drives for all Android access.

Bundle id / package name - The app's unique identifier (`com.example.target`). Called a package name on Android and a bundle id on iOS; MobFI uses them interchangeably in flags.

Container - On iOS, the app's private directory tree. Reachable over AFC only for dev-signed apps or on a jailbroken device.

Debuggable - An Android app built with `android:debuggable="true"` . Readable with `run-as` on any device, no root required.

Extraction scope - Which part of an iOS app MobFI pulls: `container` , `documents` , or `backup` .

Finding - One secret match: a rule id, a file, a line, and a redacted fingerprint.

Fingerprint (redacted) - The safe form of a matched secret: first four characters plus total length.

Keybag - The iOS structure holding the class keys that protect keychain items and backup files. Unlocked with the backup password.

Keychain - iOS credential store. Recoverable from an encrypted backup, or on a jailbroken device.

Keystore - Android credential store. Blobs live under `/data/misc/keystore` ; root required.

Protection class - How strictly a keychain item is protected (`WhenUnlocked` , `AfterFirstUnlock` , `...ThisDeviceOnly`).

run-as - The Android shell command that runs as a debuggable app's own uid.

Secure Enclave / StrongBox / TEE - Hardware-backed key storage on iOS and Android. Key material is non-exportable by design.

Simulator vs emulator - iOS *Simulators* run on macOS and keep containers on the host filesystem. Android *emulators* are full virtual devices reached through `adb` .

Structural diff - A semantic comparison (rows, fields) rather than a byte comparison, applied to SQLite, JSON, and plists.

udid - Unique Device Identifier. The iOS equivalent of an adb serial.

usbmuxd - The daemon multiplexing USB connections to iOS devices. Required on Linux; provided by Apple Mobile Device Support on Windows.

Further reading

- **OWASP Mobile Application Security Testing Guide (MASTG)** - the reference methodology for mobile testing. MobFI covers the data-storage portions.
- **OWASP Mobile Top 10** - vulnerability classes to map findings onto in a report.
- **Apple Platform Security guide** - authoritative on Keychain protection classes, keybags, and the Secure Enclave.
- **Android Keystore documentation** - authoritative on security levels and what hardware backing does and does not guarantee.
- **libimobiledevice documentation** - the tools underneath every iOS workflow here.

Handbook maintenance

This handbook is generated from per-chapter sources in `docs/handbook/`. To change it, edit the chapter, not the generated output:

```
$ $EDITOR docs/handbook/07-scanning.md
$ make handbook
```

The generated `docs/handbook.md` and `docs/handbook.pdf` are regenerated automatically when you commit a chapter change, via the repository's pre-commit hook. See `docs/handbook/README.md` for the full workflow, including how to add screenshots and how to install the PDF toolchain.